



# CS & IT ENGINEERING



## Algorithms

### Heap Algorithms

Lecture No.- 03



By- Aditya sir

# Recap of Previous Lecture



Topic

Topic

Heap Algos



# Topics to be Covered



Topic

Topic

Topic

94Qs + Practice Ques





## About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on LinkedIn where I share my insights and guide students and professionals.





Telegram Link for Aditya Jain sir: [https://t.me/AdityaSir\\_PW](https://t.me/AdityaSir_PW)

TTTe



## Topic : Heap Algorithm

#Q. In a Binary Max-Heap with  $n$  elements, the smallest element can be found in time of \_\_\_\_\_.

52%

default: WC Complexity of  
most optimal algo.

**A**  $O(n \log n)$

**B**  $O(n)$

**C**  $O(n^2)$

**D**  $O(1)$



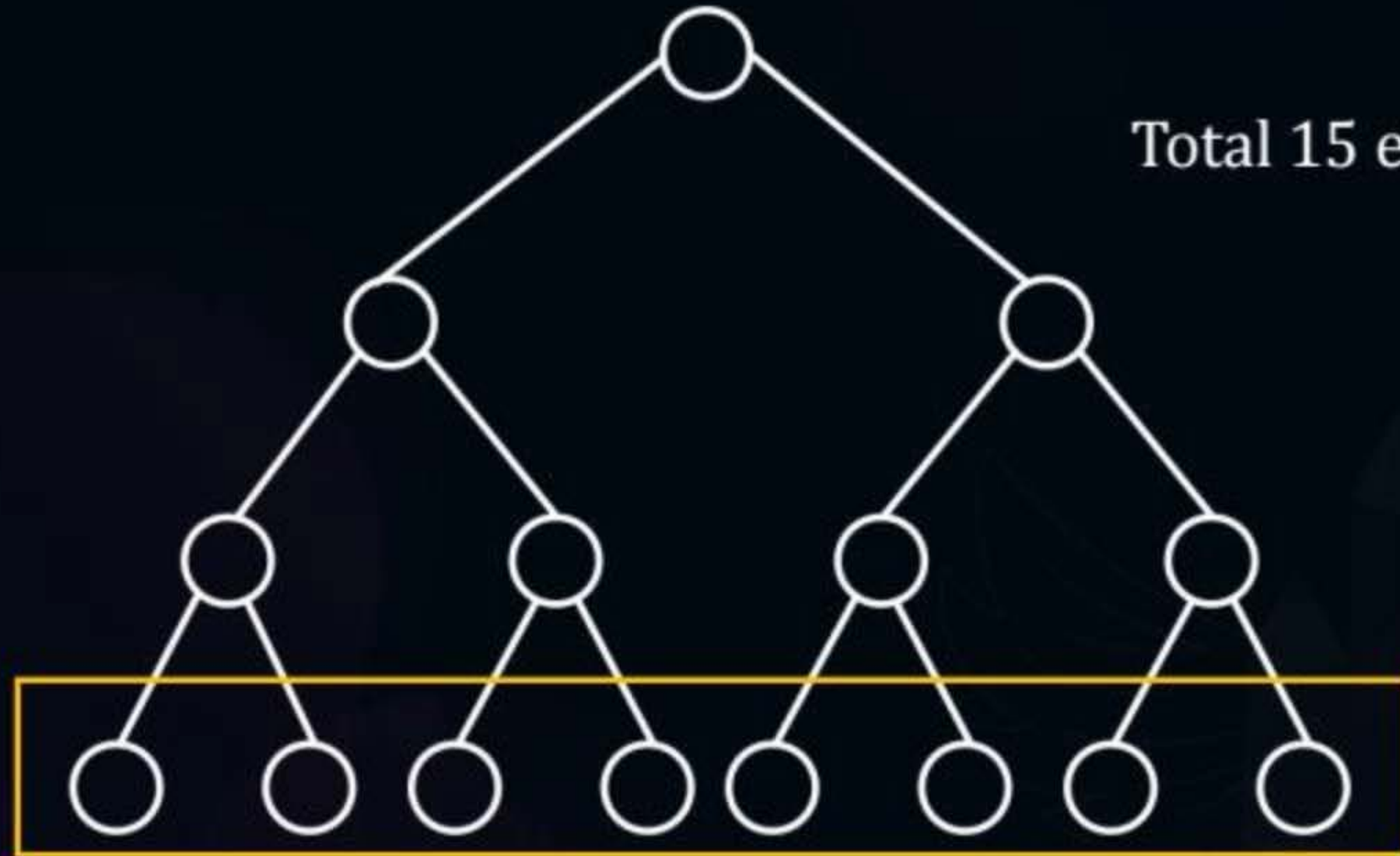
## Topic : Heap Algorithm

Max-Heap

$$= \lfloor n/2 \rfloor$$

15

≈



Total 15 elements  $\rightarrow n$

Smallest element always a leaf of max-Heap





## Topic : Heap Algorithm

15

→

8

Total

last level

$n \rightarrow \simeq \left\lceil \frac{n}{2} \right\rceil$  at leaf level

Appr. Linear search once these  $\frac{n}{2}$  elements →  $O(n)$



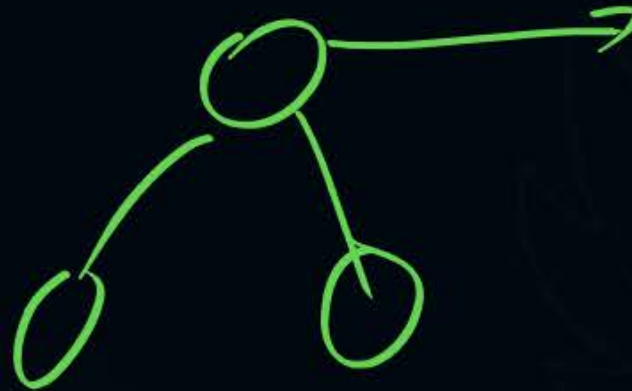


## Topic : Heap Algorithm

Appr. 2

Given max-Heap perform heapify on this to convert it to a min-heap and then print the root node-(min element).

Heapify  $\rightarrow O(n)$





## Topic : Heap Algorithm

#Q. Level order traversal of a binary max heap generates:  $\langle 10, 8, 5, 3, 2 \rangle$ . To this Heap Insert:  $\langle 1 \text{ and } 7 \rangle$ ; What is the resultant level order Traversal

91%

**A**  $10, 8, 7, 3, 5, 2, 1$  ✗

**B**  $10, 8, 7, 3, 2, 1, 5$  ✓

**C**  $10, 7, 8, 3, 5, 1, 2$  ✗

**D**  $10, 7, 8, 5, 3, 2, 1$  ✗

(B)







## Topic : Heap Operations

#Q. Given a Binary Heap with 'n' elements, and it is required to insert 'n' more elements not necessarily one after another into this heap. The total time required for this operation is \_\_\_\_\_?

24%

**A**  $O(n^2)$

**B**  $O(n \log n)$  → X

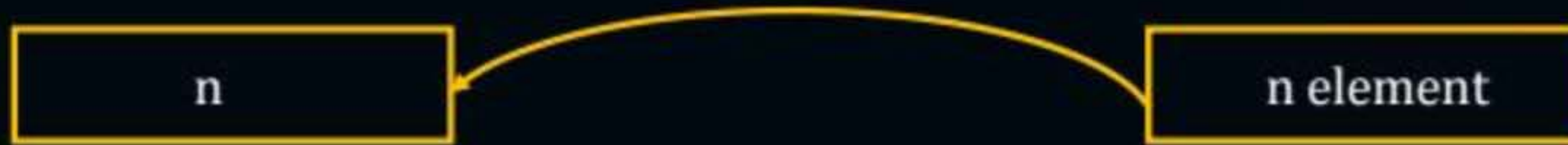
**C**  $O(n)$

**D**  $O(n^2 \log n)$

**Sol.**

Given Heap

Insert



**Appr-1.**

insertion

$n \longleftarrow 1 \longrightarrow O(\log n)$

So for n element  $\rightarrow O(n * \log n)$





Appr-2.

Heapify/ Build Heap:

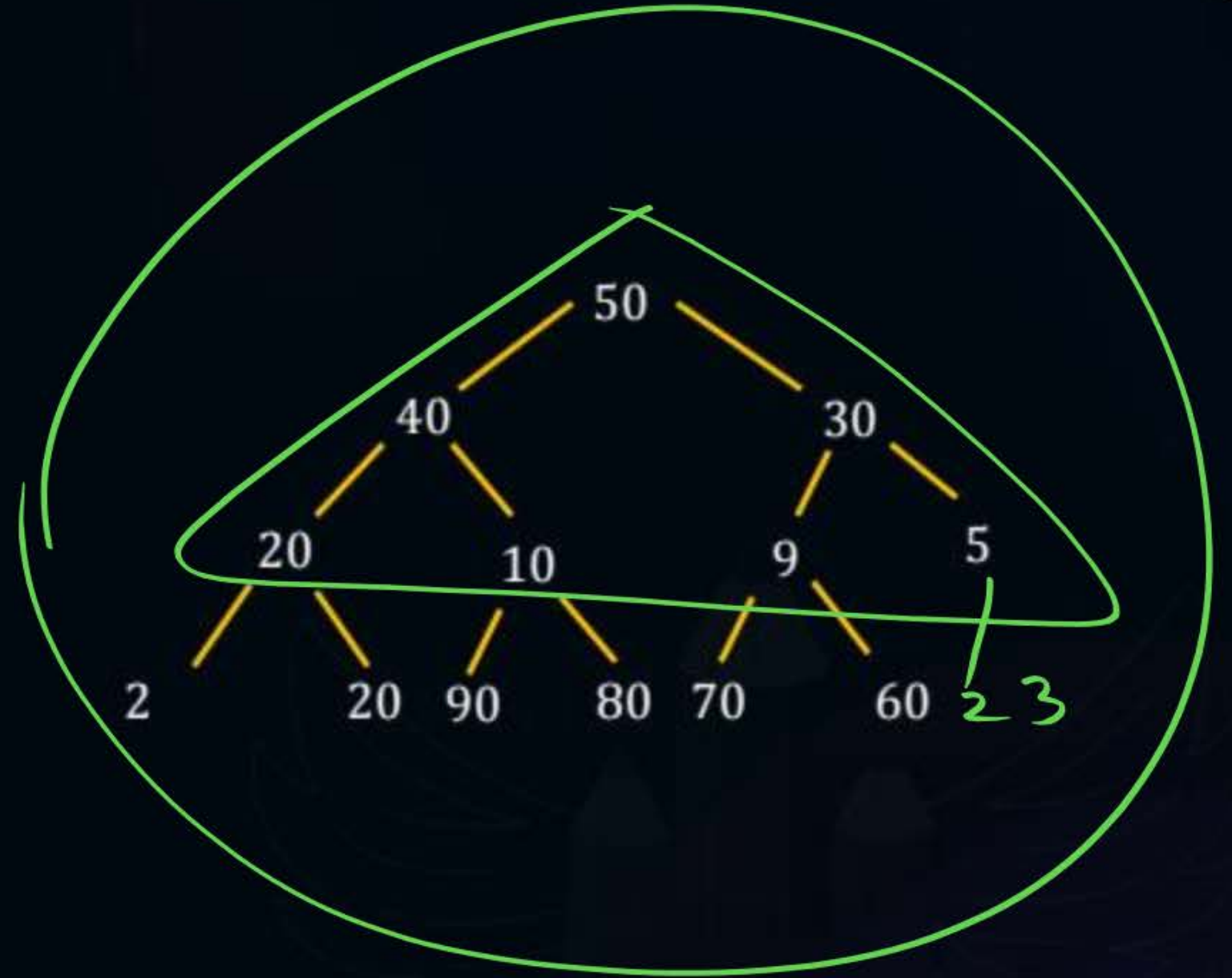
Heap



n new elements

$N \rightarrow O(N)$

$2n \rightarrow O(2n) = O(n)$





## Topic : Heap Operations

#Q. Given binary heap in array with the smallest element at the root, the 7<sup>th</sup> smallest element can be found in time complexity of \_\_\_\_\_?

30%

**A**

$O(n)$



**B**

$O(n \log n)$



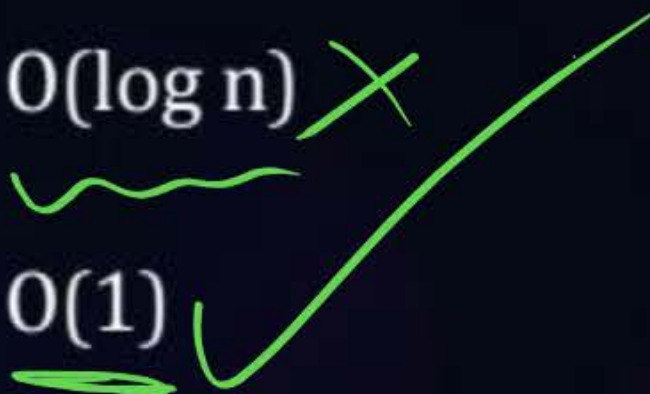
**C**

$O(\log n)$



**D**

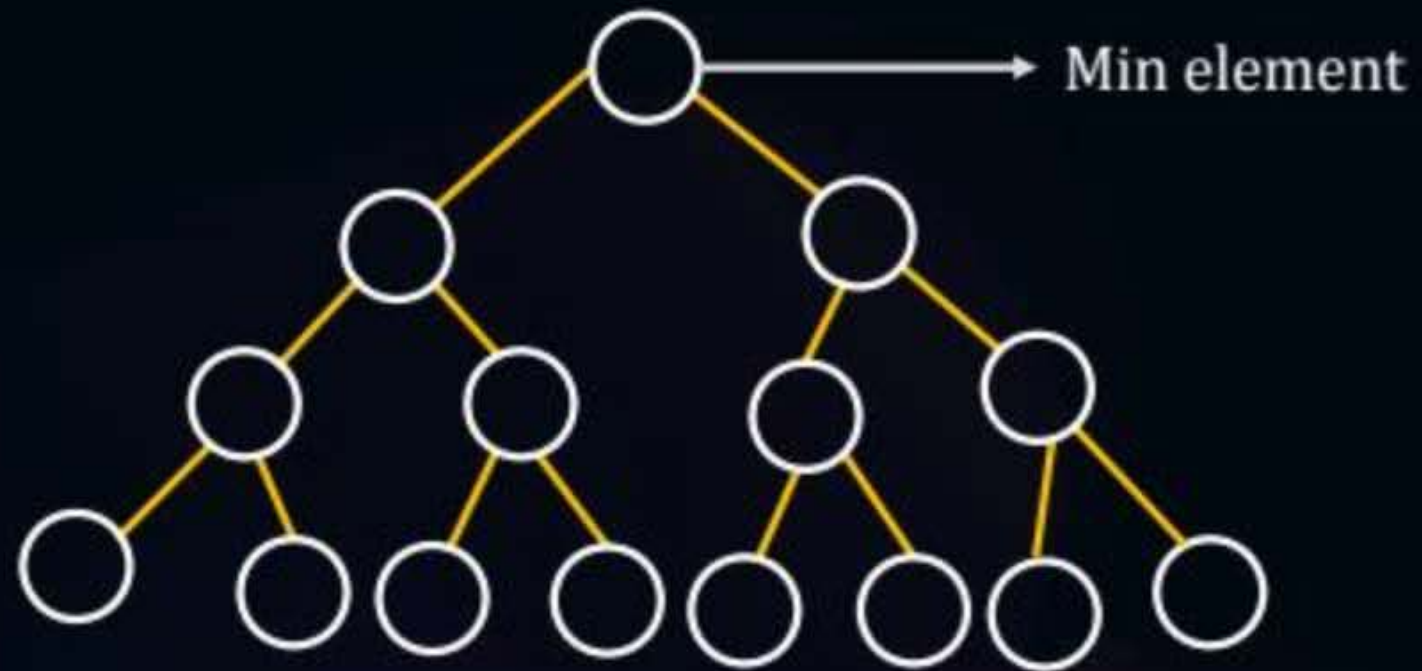
$O(1)$





Appr-1.

Usual / Traditional approach:  
Min Heap



1<sup>st</sup> min  $\rightarrow$  1 delete

2<sup>nd</sup> min  $\rightarrow$  1 delete

7<sup>th</sup> min  $\rightarrow$  7 delete

1 delete operator TC  $\rightarrow O(\log n)$

Appr-2.

Better/ non-traditional

1<sup>st</sup> min → level 1

2<sup>nd</sup> min → level 2

3<sup>rd</sup> min → level 2

→ max depth: level 3



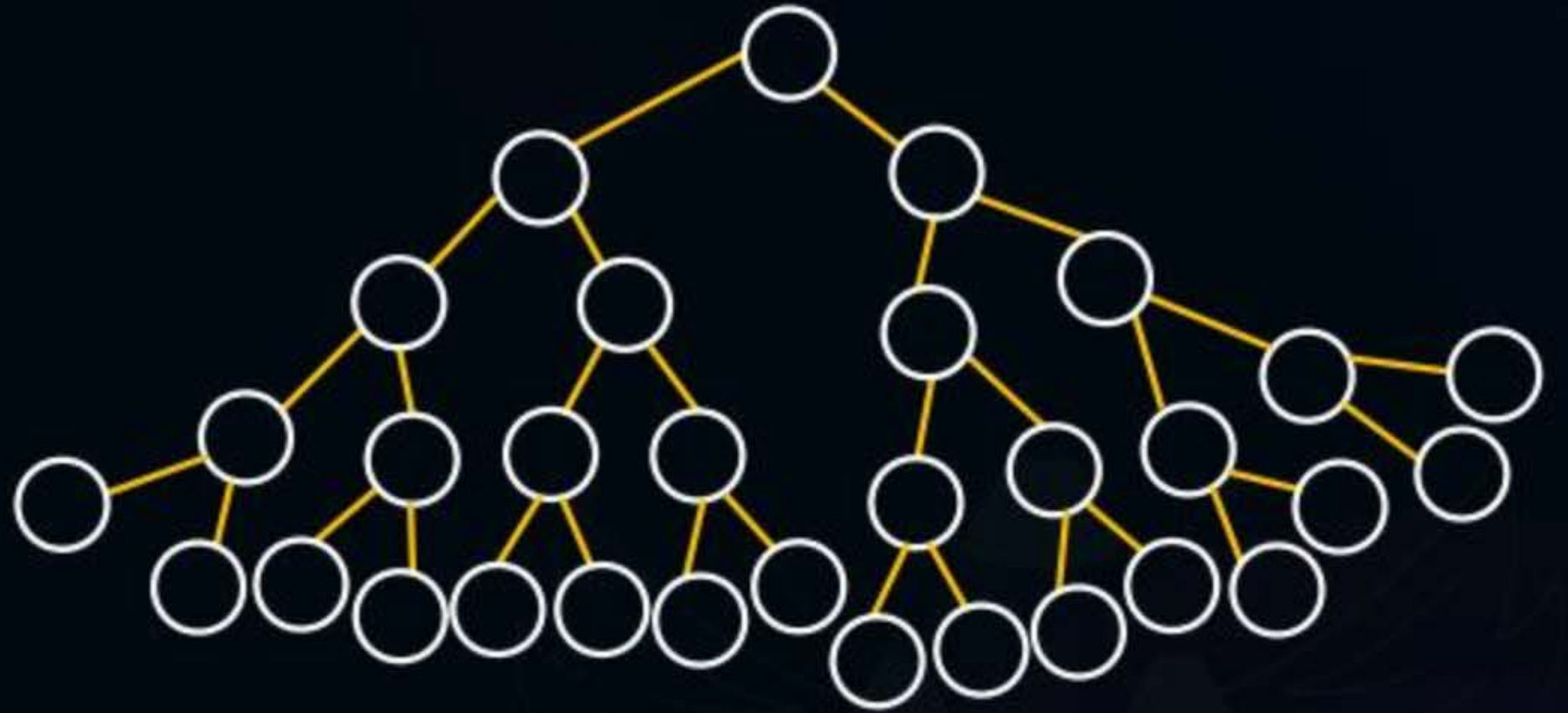
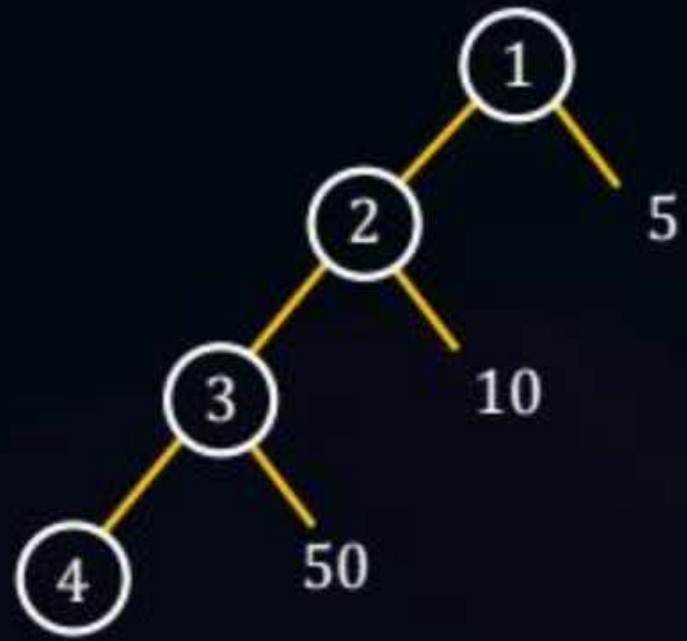


4<sup>th</sup> min  $\rightarrow$  max depth level 4.

7<sup>th</sup> min  $\rightarrow$  max depth level 7

Heap  $\rightarrow$  20 cr elements







We can be sure, that the 7<sup>th</sup> min element is present within level 1 to level 7.

1<sup>st</sup> level  $\rightarrow$  max 1 elements  $\rightarrow 2^0$

2<sup>nd</sup> level  $\rightarrow$  max 2 elements  $\rightarrow 2^1$

3<sup>rd</sup> level  $\rightarrow$  max 4 elements  $\rightarrow 2^2$

⋮

i<sup>th</sup> level  $\rightarrow$  max  $2^{i-1}$  elements

L1  $\rightarrow 2^0$

L2  $\rightarrow 2^1$

L3  $\rightarrow 2^2$

L4  $\rightarrow 2^3$

L5  $\rightarrow 2^4$

L6  $\rightarrow 2^5$

L7  $\rightarrow 2^6$

So require element present within first 7 levels

$$2^0 + 2^1 + 2^2 \dots 2^6 = 2^7 - 1$$

Min Heap  $\rightarrow$   $n = 200$  cr elements  
still explore :  $2^7 - 1$

'n'  $\rightarrow$   $2^7 - 1$  (const)

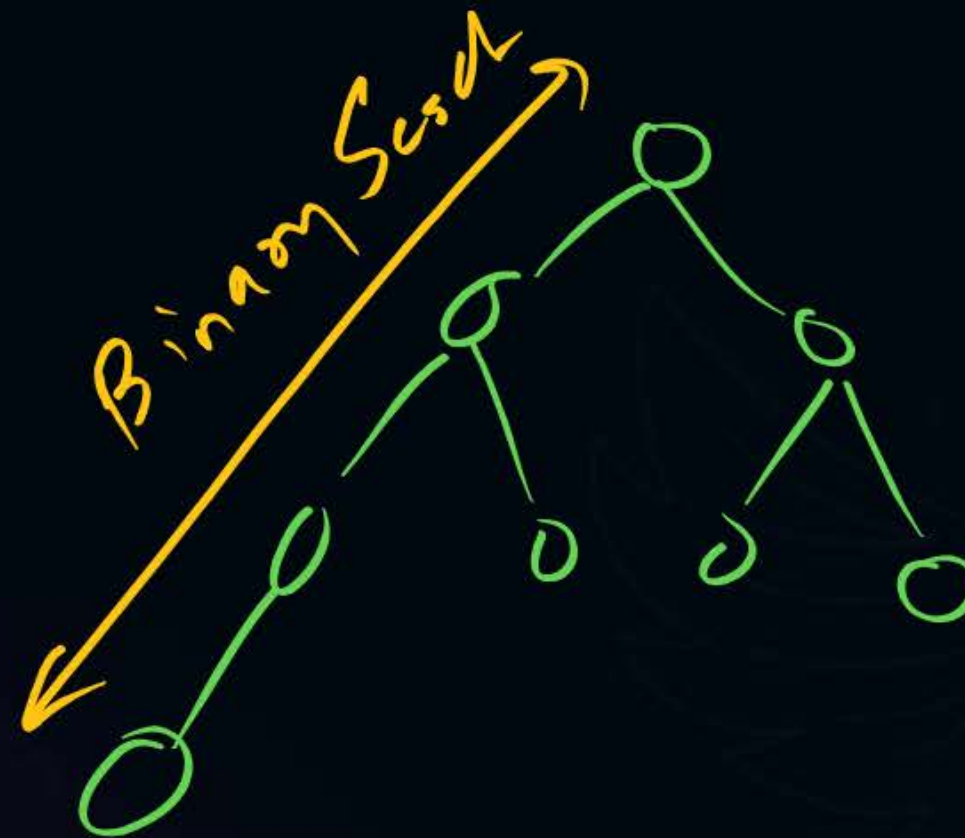


## Topic : Heap Operations



#Q. Consider binary Heap in an Array with  $n$  elements. It is desired to insert an element into the Heap. If a binary search is performed along the path from newly inserted element to the root then the number of comparisons made is order of \_\_\_\_\_.

- A**  $O(\log n)$
- B**  $O(n)$
- C**  $O(\log(\log n))$
- D**  $O(1)$



46%

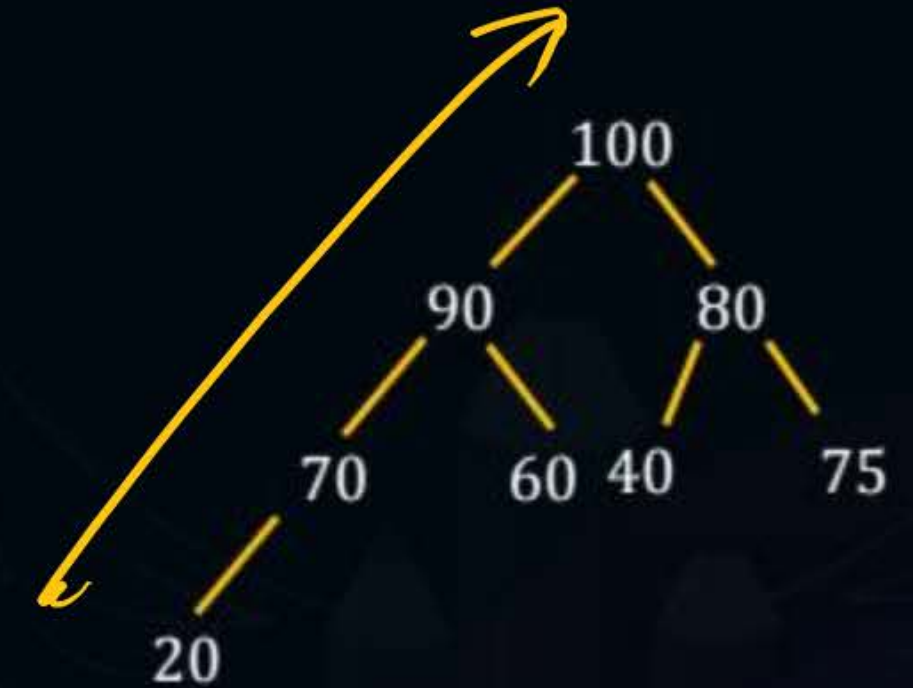


**Sol.**

max number of elements in the path from new node to root if Heap has  $n$  elements ?

$n$  elements  $\rightarrow O(\log n)$

$\log n$  elements  $\rightarrow O(\log(\log n))$





## Topic : Heap Operations



#Q. The approximate number of element that can be sorted in  $O(\log n)$  time using Heap sort is \_\_\_\_\_.

75%

29%

0/0  
😊

**A**

~~$O(n)$~~  →

**B**

~~$O(1)$~~

**C**

$O\left(\frac{\log n}{\log(\log n)}\right)$  ✓

**D**

~~$O(\log n)$~~  →

**Sol.**

We know,  $n$  element can be sorted in  $O(n \log n)$  time using Heap sort.

' $n$ ' elements  $\rightarrow O(n \log n)$  time.

**Check option D:**

no. of elements  $n$

$O(\log n)$

$\rightarrow$

$O(n \log n)$

$\rightarrow$

$O(\log n * \log(\log n))$





logic.

to sort n element  $\rightarrow$  time  $O(n \log n)$

Check option C:

$$x = \frac{\log n}{\log(\log n)}$$

$$x \rightarrow n \log n$$

$$= \left\lceil \frac{\log n}{\log(\log n)} \right\rceil * \log \left\lceil \frac{\log n}{\log(\log n)} \right\rceil$$

*dominating*

$$= \left( \frac{\log n}{\log(\log n)} * [\log(\log n) - \log(\log(\log n))] \right)$$

$$= \log n - \frac{\log n * \log \log \log n}{\log \log n}$$

$$= O(\log n)$$

$$n - \frac{n * \log \log n}{\log n} < 1$$



## Topic : Heap Operations

#Q. The minimum number of interchanges needed to convert the array into a max-heap is

89, 19, 40, 17, 12, 10, 2, 5, 7, 11, 6, 9, 70

77%

**A**

0

**B**

1

**C**

2

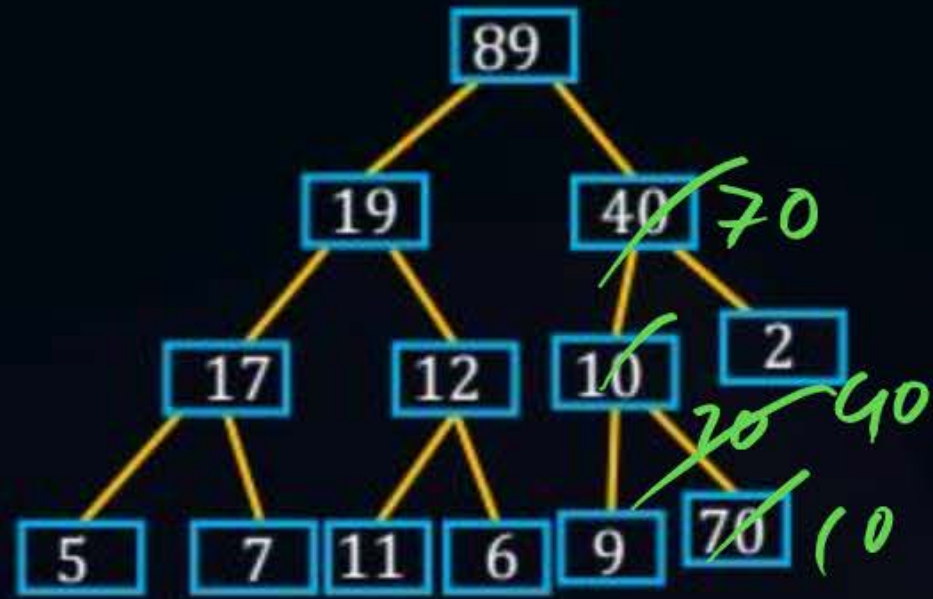
**D**

3



**Sol.**

Given:



Heapify



How: insertion  
method.

10  $\longleftrightarrow$  70 ①  
40  $\longleftrightarrow$  70 ②

Interchange

② ✓





## Topic : Heap Operations



#Q. An array of integers of size  $n$  can be converted into a heap by adjusting the heaps rooted at each internal node of the complete binary tree starting at the node  $\lfloor (n-1)/2 \rfloor$  and doing this adjustment up to the root node (root node is at index 0) in the order  $\lfloor (n-1)/2, \lfloor (n-3)/2 \rfloor, \dots, 0$ . The time required to construct a heap in this manner is

**A**

$O(\log n)$

**B**

$O(n)$

**C**

$O(n \log \log n)$

**D**

$O(n \log n)$

Heapify  $\rightarrow O(n)$

**Sol.**

This is formal explanation process of build Heap / Heapify process.  
 $n \rightarrow O(n)$





## Topic : Heap Operations



#Q. An array  $X$  of  $n$  distinct integers is interpreted as a complete binary tree. The index of the first element of the array is 0. If only the root node does not satisfy the heap property, the algorithm to convert the complete binary tree into a heap has the best asymptotic time complexity of

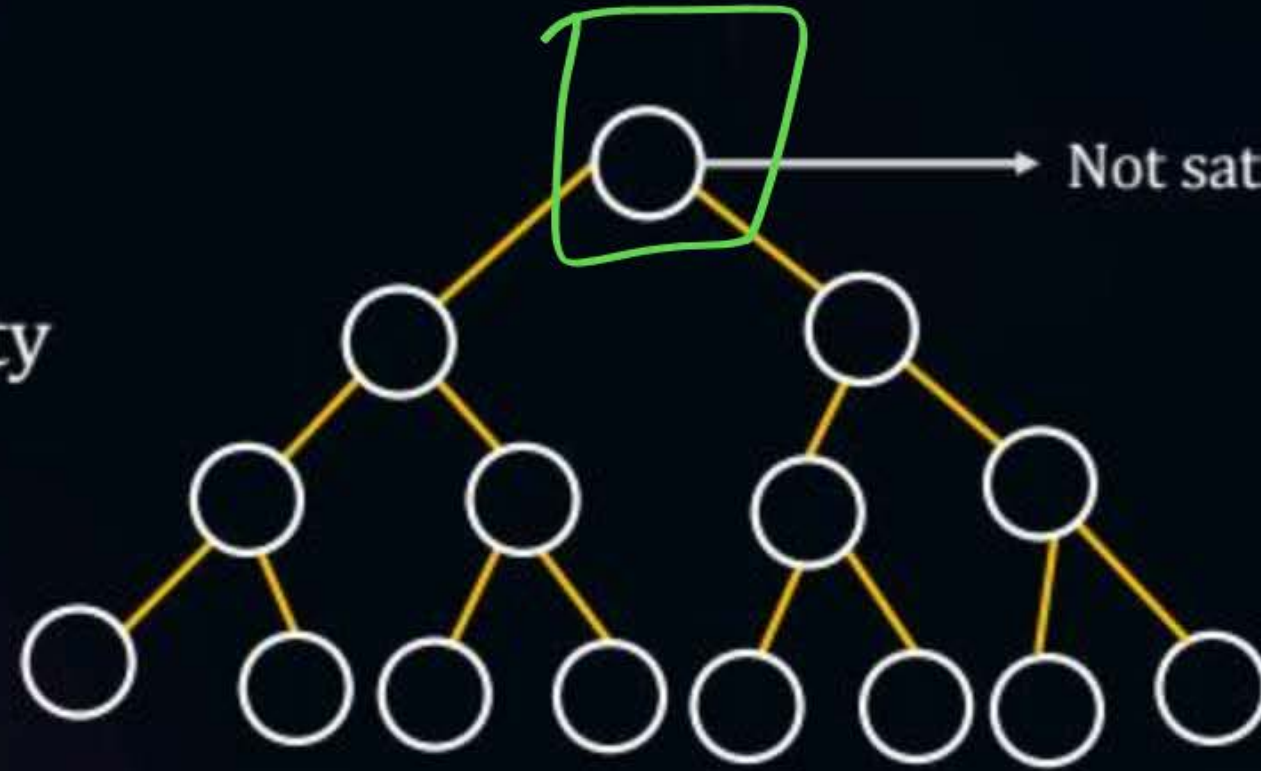
- A**  $O(n)$
- B**  $O(\log n)$
- C**  $O(n \log n)$
- D**  $O(n \log \log n)$



**Sol.**

best asymptotic complexity

Best  $\begin{pmatrix} WC1 \\ WC2 \\ WC3 \end{pmatrix}$



Not satisfy Heap property.

1. Appr 1: Heapify :  $O(n)$

2. Appr 2: adjust root node:  $\rightarrow \log(n)$

*adjust*



## Topic : Heap Operations

#Q. Consider a max heap, represented by the array:  
40, 30, 20, 10, 15, 16, 17, 8, 4

| Array index | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8 | 9 |
|-------------|----|----|----|----|----|----|----|---|---|
| Value       | 40 | 30 | 20 | 10 | 15 | 16 | 17 | 8 | 4 |

Now consider that a value 35 is inserted into this heap. After insertion, the <sup>new</sup> heap is

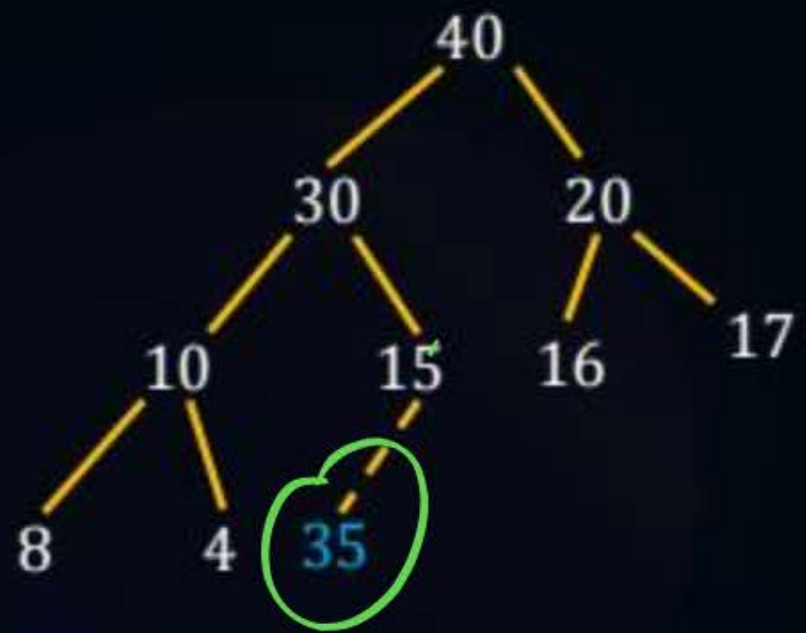
- A** 40, 30, 20, 10, 15, 16, 17, 8, 4, 35 ~~X~~
- B** 40, 35, 20, 10, 30, 16, 17, 8, 4, 15 ✓
- C** 40, 30, 20, 10, 35, 16, 17, 8, 4, 15 ~~X~~
- D** 40, 35, 20, 10, 15, 16, 17, 8, 4, 30 ~~X~~

93%



**Sol.**

40 35 20 10 30 16 17 8 4 15







## 2 mins Summary



Topic

Topic

Topic

Topic

Topic

Heap Algo

PYB + Practice



**THANK - YOU**